Assignment 2. OOP Project

Kyojin Park

May 2026

1 Introduction

Drug–target interaction analysis is an important task in biomedical research and drug discovery. In the early stages of drug development, researchers need to understand how a drug molecule interacts with a target protein and how strong the interaction is. This information can help identify promising drug candidates and reduce the time and cost required for experimental screening.

Drug–target interaction data generally consists of several related components, such as drug information, protein information, interaction pairs, and binding affinity scores. If these components are handled only with simple variables, lists, or dictionaries, the code can become difficult to manage as the system grows. For example, adding a new prediction method, storing multiple interaction results, or comparing different drug–target pairs may require repeated modifications across different parts of the code.

Object-oriented programming provides a useful way to organize and manage this type of complex data. By representing each real-world entity as an object, such as Drug, Protein, Interaction, and PredictionResult, the system can clearly separate the responsibilities of each component. Each class stores the data it needs and provides methods related to its own role. As a result, the program becomes easier to understand, maintain, and extend.

The objective of this project is to implement a system that manages drug–target interaction information using an object-oriented structure. The proposed system manages drug and protein information, creates drug–target interaction objects, predicts or stores affinity scores, and reports the analysis results. Through this implementation, this project demonstrates how core OOP concepts such as class, object, encapsulation, composition, inheritance, and polymorphism can be applied to a practical data analysis problem related to drug discovery.

The focus of this project is not to build a high-performance deep learning model, but to design a clear and extensible object-oriented software structure using a small sample dataset.

2 Related Work

Drug–target interaction analysis is used to understand the interactions between drugs and proteins and to examine how strongly a specific drug binds to a target protein. In the drug discovery process, it is important to identify promising compounds among many candidates, which makes the analysis of drug–target interactions and binding affinity highly important. Binding affinity generally refers to the strength of the binding between a drug and a protein.

In drug–target interaction data, drugs are commonly represented as SMILES strings. SMILES is a method of expressing the structure of a molecule in the form of a string, allowing computers to process the chemical structure of a drug. Proteins are generally represented as amino acid sequences, which describe the order of amino acids that make up the protein. Therefore, drug–target interaction data usually consists of a drug's SMILES, a protein's sequence, and the affinity score between them.

Representative public drug–target interaction datasets include the Davis dataset, the KIBA dataset, and BindingDB. Davis and KIBA are widely used benchmark datasets for drug–target affinity prediction research, while BindingDB provides a variety of experimentally measured drug–target binding data. These datasets show that an actual drug–target interaction analysis system should include data elements such as drugs, proteins, interactions, and affinity scores.

Existing drug–target interaction prediction studies mainly use deep learning models to learn representations of drugs and proteins and to predict binding affinity. For example, a drug's SMILES may be processed as a 1D sequence or converted into a molecular graph, while a protein sequence may be used as input to CNN- or Transformer-based models. However, the purpose of this project is not to implement a high-performance prediction model, but to design a software structure that can manage these data elements in an object-oriented manner.

In this project, these data elements are not used for model training, but are organized through object-oriented classes such as Drug, Protein, Interaction, and PredictionResult.

3 Proposed OOP System

This project proposes an object-oriented analysis system for managing drug–target interaction information. The purpose of this system is not to train a deep learning model or to predict new affinity scores. Instead, the project focuses on designing a software structure that represents and manages drugs, proteins, interactions, and prediction results as objects using a sample dataset that already contains affinity scores.

3.1 System Overview

This system uses three CSV files: sample_drugs.csv, sample_proteins.csv, and sample_interactions.csv. These files provide the data needed to create Drug, Protein, Interaction, and PredictionResult objects. The main design goal is to separate drug information, protein information, interaction data, and result management into different classes.

3.2 Main Classes

Drug stores drug information such as drug ID, CID, and SMILES strings, and provides a short_smiles() method for display purposes. Protein stores protein attributes including accession number, gene name, and amino acid sequence, and provides a sequence_length() method. Interaction contains a Drug object and a Protein object as attributes, directly linking the two entities. The pair_id property generates an identifier such as D2-P63. PredictionResult stores an interaction with its affinity score and provides an affinity_level() method that classifies the result as High (8.0), Medium (6.0–7.9), or Low (¡6.0). DataLoader reads the CSV files and converts rows into Drug, Protein, and Interaction objects. It also builds an affinity lookup table keyed by drug–protein pairs. AffinityPredictor is an abstract base class that defines the predict() interface using ABC and abstractmethod, ensuring all predictor subclasses share a consistent method signature. StoredAffinityPredictor retrieves affinity scores directly from sample_interactions.csv and wraps them in PredictionResult objects, without performing any new predictions. RuleBasedAffinityPredictor calculates scores using a simple formula based on SMILES length and protein sequence length. It is included solely to demonstrate polymorphism, and its scores do not represent actual biological affinity. ReportManager organizes PredictionResult objects and provides methods such as average_affinity(), best_result(), filter_by_level(), and print_summary().

3.3 OOP Concepts Applied in the System

This system applies several object-oriented programming concepts.

First, class and object are used to represent the main elements of the system. The Drug, Protein, Interaction, and PredictionResult classes represent drug information, protein information, drug–target pairs, and analysis results as objects. This makes the data structure clearer than using only lists or dictionaries.

Second, encapsulation is applied by grouping data and related methods within the same class. For example, the Protein class stores the protein sequence and provides the sequence_length() method. Similarly, the PredictionResult class stores an affinity score and provides the affinity_level() method.

Third, composition is used in the Interaction class. Each Interaction object contains one Drug object and one Protein object. This structure directly represents the relationship between a drug and a target protein.

Fourth, abstraction is applied through the AffinityPredictor class. This class defines the common predict() interface, but does not implement a specific prediction method. The detailed behavior is left to its child classes.

Fifth, inheritance is used by StoredAffinityPredictor and RuleBasedAffinityPredictor. Both classes inherit from AffinityPredictor and implement the required predict() method in their own way. This makes it possible to add new predictor classes later without changing the overall structure.

Sixth, polymorphism is demonstrated through the two predictor classes. Both predictors use the same predict() method name, but they behave differently. StoredAffinityPredictor retrieves existing affinity scores from the CSV file, while RuleBasedAffinityPredictor uses a manually defined formula only to demonstrate polymorphism. Therefore, the same method call can produce different results depending on the actual predictor object.

4 Implementation & Usage / Results

This section describes how the proposed OOP-based Drug–Target Interaction Analysis System works in practice. The system is implemented in Python and uses three CSV files as input: `sample_drugs.csv`, `sample_proteins.csv`, and `sample_interactions.csv`.

4.1 Implementation Environment

The system is implemented using Python's `dataclass`, `csv`, `ABC`, and `abstractmethod`. The `dataclass` decorator is used to define domain objects such as `Drug`, `Protein`, `Interaction`, and `PredictionResult` concisely, while `ABC` and `abstractmethod` are used to define the common interface for all predictor classes.
Since `sample_interactions.csv` already contains affinity scores for each drug–target pair, the system does not train a deep learning model or predict new affinity values. Instead, it focuses on managing the existing data within an object-oriented structure.

4.2 Execution Workflow

The overall execution flow of the system is as follows.

`CSV files` → `DataLoader` → `Drug/Protein objects` → `Interaction objects` → `StoredAffinityPredictor` → `PredictionResult objects` → `ReportManager` → `Summary`

`DataLoader` reads the CSV files and creates `Drug` and `Protein` objects. It then links each drug and protein using the `Drug_Index` and `Protein_Index` from the interaction data to construct `Interaction` objects.
Next, `StoredAffinityPredictor` retrieves the affinity score for each interaction and wraps it in a `PredictionResult` object. Finally, `ReportManager` organizes all results and prints the summary.

4.3 Key Code Snippets

(1) Object Creation and Prediction

The following code shows the overall flow of loading CSV data, converting it into objects, and generating prediction results.

```
loader = DataLoader(data_dir)
drugs = loader.load_drugs()
proteins = loader.load_proteins()
affinity_table = loader.load_affinity_table()
interactions = loader.load_interactions(drugs, proteins)

predictor: AffinityPredictor = StoredAffinityPredictor(
    affinity_table)
results = [predictor.predict(interaction) for interaction in
     interactions]

report = ReportManager(results)
report.print_summary()
```

`StoredAffinityPredictor` calls `predict()` on every `Interaction` object and converts each one into a `PredictionResult` object. `ReportManager` then uses the resulting list to compute and display the average affinity, the best interaction, and the detailed results.

(2) StoredAffinityPredictor — Encapsulation and Abstraction

```
class StoredAffinityPredictor(AffinityPredictor):
    def __init__(self, affinity_table: Dict[tuple[int, int],
         float]):
        self._affinity_table = affinity_table

    def predict(self, interaction: Interaction) ->
        PredictionResult:
        key = (interaction.drug.drug_id, interaction.protein
            .protein_id)
        if key not in self._affinity_table:
            raise ValueError(
                f"No␣affinity␣score␣exists␣for␣{interaction.
                    pair_id}"
            )
        score = self._affinity_table[key]
        return PredictionResult(interaction=interaction,
            affinity_score=score)
```

`self._affinity_table` encapsulates the affinity scores loaded from the CSV file inside the predictor. The `predict()` method constructs a key from the drug ID and protein ID of the given `Interaction` object, retrieves the corresponding score, and returns a `PredictionResult` object.

(3) Polymorphism — RuleBasedAffinityPredictor

```
# Both declared as AffinityPredictor type
predictor: AffinityPredictor = StoredAffinityPredictor(
    affinity_table)
demo_predictor: AffinityPredictor =
    RuleBasedAffinityPredictor()

# Same predict() call, different behavior
result = predictor.predict(interaction)      # Returns
    stored CSV value
demo_result = demo_predictor.predict(interaction)  #
    Computes via formula
```

Although both predictors are declared as `AffinityPredictor` type, calling `predict()` on each produces different results. This demonstrates polymorphism, where the same interface supports different behaviors depending on the actual class of the object.

4.4 Execution Results

The system processed a total of 14 drug–target interactions.

```
Drug-Target Interaction Analysis Summary
============================================================
Number of results: 14
Average affinity: 6.679
Best interaction: D2-P63 (CDC2L5) Affinity=9.027
```

The affinity level classification criteria are as follows.

```
Affinity >= 8.0        -> High
6.0 <= Affinity < 8.0  -> Medium
Affinity < 6.0         -> Low
```

Among the 14 interactions, 4 were classified as High (D2-P13, D2-P63, D6-P430, D9-P35), 5 as Medium, and 5 as Low. The interaction with the highest affinity score was `D2-P63`, corresponding to the target protein `CDC2L5` with an affinity score of 9.027.

4.5 Polymorphism Example

The following output was produced using `RuleBasedAffinityPredictor` to demonstrate polymorphism.

```
Polymorphism Example : Using a simple artificial formula
------------------------------------------------------------
Using a simple artificial formula D0-P0:  Affinity=5.800, Level=Low
Using a simple artificial formula D0-P1:  Affinity=6.000, Level=Medium
...
```

These results are not actual biological predictions. `RuleBasedAffinityPredictor` computes scores using a simple artificial formula based on the length of the SMILES string and the protein sequence length. Comparing these values with the stored affinity scores in Section 4.4 confirms that the same `predict()` call produces different results depending on which predictor object is used, which is the essence of polymorphism.

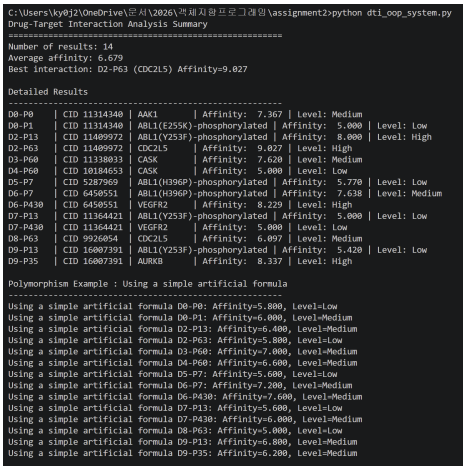

Figure 1: Terminal output of the Drug–Target Interaction Analysis System.

4.6 Discussion

The execution results confirm that the proposed system correctly reads CSV data and manages drugs, proteins, interactions, and prediction results as objects. The responsibilities of data loading, object creation, affinity score retrieval, and result reporting are clearly separated into individual classes. This structure makes the codebase easy to understand and maintain, and allows new predictors or additional analysis features to be added in the future without modifying the existing code.

5 Limitations

The current implementation focuses on object-oriented structure design, and several limitations exist in its present form.

First, the system does not perform actual affinity prediction. `StoredAffinityPredictor` simply retrieves affinity scores that are already stored in the CSV file and does not support predicting affinity values for new drug–target pairs. Since predicting novel interactions using trained models is a central challenge in real drug discovery research, this is the most significant limitation of the current system.

Second, the dataset used in this project is small in scale. Rather than using a complete dataset, only a subset of drugs, proteins, and interactions was extracted to form a sample dataset. While this is well-suited for demonstrating the system's behavior in a clear and straightforward manner, it does not fully reflect the large-scale Drug–Target Interaction data used in actual drug development research.

Third, `RuleBasedAffinityPredictor` is intended solely as an example to illustrate polymorphism and is not a real biological prediction model. This class computes affinity scores using a simple artificial formula based on the length of the SMILES string and the protein sequence length. Therefore, its output does not carry actual biological meaning and should not be interpreted as a real analysis result.

For future improvements, several directions can be considered. New predictor classes such as `DeepLearningPredictor` or `GraphBasedPredictor` could be implemented to integrate real machine learning or deep learning models into the system, while preserving the existing `AffinityPredictor` structure. In addition, the system could be extended to support larger datasets or connected to a database for more systematic management of drug, protein, and interaction information. Adding a GUI or visualization features to `ReportManager` would also make it easier for users to interpret and explore the results.

6 Conclusion

This project implemented an object-oriented system in Python for managing drug–target interaction data. The primary goal was not to build a high-performance prediction model, but to design a software structure that represents real-world entities such as drugs, proteins, interactions, and prediction results as clearly defined classes, each with its own distinct responsibility.

Throughout the implementation, the core OOP concepts - classes and objects, encapsulation, composition, abstraction, inheritance, and polymorphism - were applied to a practical problem in the domain of drug discovery. In particular, designing `AffinityPredictor` as an abstract base class and implementing `StoredAffinityPredictor` and `RuleBasedAffinityPredictor` as its subclasses demonstrated how the same interface can support different behaviors depending on the object being used.

The most important lesson learned from this project is the difference between managing data as simple lists or dictionaries and representing it as objects. By grouping each class's data together with its related behavior, the role of each component becomes clear, and the system can be extended with minimal modification to existing code. These design principles are expected to serve as a valuable foundation for developing more complex software systems in the future.

References

[1] M. I. Davis, J. P. Hunt, S. Herrgard, P. Ciceri, L. M. Wodicka, G. Pallares, M. Hocker, D. K. Treiber, and P. P. Zarrinkar, "Comprehensive analysis of kinase inhibitor selectivity," *Nature Biotechnology*, vol. 29, pp. 1046–1051, 2011.

[2] H. Öztürk, A. Özgür, and E. Ozkirimli, "DeepDTA: deep drug–target binding affinity prediction," *Bioinformatics*, vol. 34, no. 17, pp. i821–i829, 2018.

[3] H. Öztürk, A. Özgür, and E. Ozkirimli, *DeepDTA GitHub Repository: Drug–Target Binding Affinity Prediction Dataset*. Available: `https://github.com/hkmztrk/DeepDTA`

[4] J. Abramson, J. Adler, J. Dunger, et al., "Accurate structure prediction of biomolecular interactions with AlphaFold 3," *Nature*, vol. 630, pp. 493–500, 2024. doi: `10.1038/s41586-024-07487`

[5] Python Software Foundation, *Python 3 Documentation: Classes*. Available: `https://docs.python.org/3/tutorial/classes.html`

[6] Python Software Foundation, *Python 3 Documentation: abc - Abstract Base Classes*. Available: `https://docs.python.org/3/library/abc.html`